

DAY 3 – AWS EC2 + DOCKER DEPLOYMENT

Learning Objective

By the end of this session, students will be able to:

- Understand AWS EC2 and cloud computing basics
 - Create and configure an EC2 Ubuntu server
 - Connect to EC2 using SSH
 - Work with basic Linux commands
 - Install Docker on EC2
 - Deploy a React application using Docker
 - Build a Docker image directly inside EC2
 - Build a Docker image outside EC2
 - Push a Docker image to Docker Hub
 - Pull a Docker image into EC2
 - Run a Docker container inside EC2
 - Access the deployed application through the EC2 Public IP
 - Understand the difference between manual deployment and image-based deployment
-

PART 1 – AWS EC2 INTRODUCTION

What is Cloud Computing?

Cloud computing is the delivery of computing resources such as servers, storage, databases, networking, and software over the internet.

Instead of purchasing and maintaining a physical server, we can create a virtual server in the cloud.

Traditional Deployment

Physical Server



Install Operating System



Install Software



Deploy Application

Cloud Deployment

AWS Cloud



Create EC2 Instance



Install Software



Deploy Application

PART 2 – WHAT IS AWS?

AWS stands for:

Amazon Web Services

AWS provides cloud services for:

- Computing
- Storage
- Networking
- Databases
- Security
- Monitoring
- DevOps
- AI/ML

For today's practical, we will mainly use:

Amazon EC2

PART 3 – WHAT IS EC2?

EC2 stands for:

Elastic Compute Cloud

Amazon EC2 allows us to create virtual servers in the AWS cloud.

An EC2 instance can be used to:

- Host websites
- Run APIs

- Run Docker containers
- Run databases
- Install development tools
- Run Jenkins
- Deploy applications

Think of EC2 as:

A virtual Linux computer running inside AWS.

PART 4 – IMPORTANT EC2 CONCEPTS

Concept	Description
Instance	Virtual server
AMI	Operating system image
Instance Type	CPU and memory configuration
Key Pair	Used for secure SSH access
Security Group	Virtual firewall
Public IPv4	Address used to access the server
EBS	Storage attached to the instance
Region	Geographic AWS location
Availability Zone	Isolated location within a region

PART 5 – CREATE AN EC2 INSTANCE

Go to:

AWS Console → EC2 → Instances → Launch Instance

Configure the insta

Operating System

Select:

Ubuntu

Instance Type

Select an instance type eligible for the Free Tier benefits available to your AWS account.

Key Pair

Create/select:

react-deployment

The key file should be:

react-deployment.pem

Security Group

Allow the following inbound traffic:

Type	Port	Purpose
SSH	22	Connect to EC2
HTTP	80	Access the web application

Launch the instance.

PART 6 – FIND THE EC2 PUBLIC IP

After the instance starts, open the instance details.

Find:

Public IPv4 address

Example:

13.xxx.xxx.xxx

This IP will be used to:

1. Connect to EC2
2. Access the deployed React application

PART 7 – CONNECT TO EC2 USING SSH

Windows PowerShell

Go to the Downloads folder where the `.pem` file is stored.

```
cd $HOME\Downloads
```

Check the key:

```
dir react-deployment.pem
```

Connect to EC2:

```
ssh -i "react-deployment.pem" ubuntu@<EC2-PUBLIC-IP>
```

Example:

```
ssh -i "react-deployment.pem" ubuntu@13.xxx.xxx.xxx
```

If prompted:

Are you sure you want to continue connecting?

Enter:

yes

After successful connection:

```
ubuntu@ip-172-31-xx-xx:~$
```

You are now inside the EC2 Ubuntu server.

PART 8 – VERIFY THE EC2 USER

Run:

```
whoami
```

Expected output:

```
ubuntu
```

We will continue using the `ubuntu` user.

Administrative commands will use `sudo`.

PART 9 – BASIC LINUX COMMANDS

Check current directory:

`pwd`

List files:

`ls`

Detailed listing:

`ls -la`

Check system information:

`uname -a`

Check disk usage:

`df -h`

Check memory:

`free -h`

PART 10 – UPDATE UBUNTU

Run inside EC2:

`sudo apt update -y`

`sudo` allows the current user to perform administrative operations.

PART 11 – INSTALL GIT

Install Git:

```
sudo apt install git -y
```

Verify:

```
git --version
```

PART 12 – INSTALL DOCKER

Install Docker:

```
sudo apt install docker.io -y
```

Check Docker:

```
docker --version
```

Example:

Docker version 29.x.x

PART 13 – CHECK DOCKER SERVICE

Run:

```
sudo systemctl status docker
```

The status should show:

active (running)

Press:

q

to exit.

PART 14 – CONFIGURE DOCKER PERMISSIONS

The **ubuntu** user may initially not have permission to communicate with Docker.

Run:

```
sudo usermod -aG docker ubuntu
```

Now exit the EC2 session:

```
exit
```

Reconnect from Windows PowerShell:

```
ssh -i "react-deployment.pem" ubuntu@<EC2-PUBLIC-IP>
```

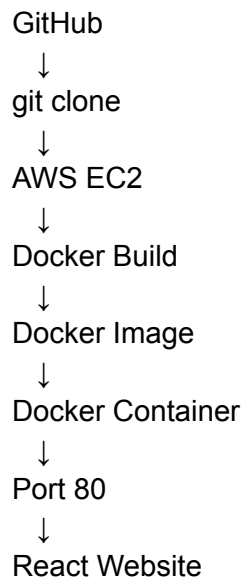
Test:

```
docker ps
```

If the command works without `sudo`, Docker permissions are configured correctly.

METHOD 1 – BUILD AND RUN DOCKER DIRECTLY INSIDE EC2

Deployment Architecture



PART 15 – CLONE THE REACT PROJECT

Inside EC2:

```
git clone https://github.com/MaX-NeO/aws-cicd.git
```

Check the files:

ls

You should see:

aws-cicd

PART 16 – ENTER THE PROJECT DIRECTORY

cd aws-cicd

Check the project files:

ls

You should see the project files including:

Dockerfile
package.json
src
...

The **Dockerfile** is required for building the Docker image.

PART 17 – BUILD THE DOCKER IMAGE INSIDE EC2

Run:

docker build -t reactapp .

Explanation:

docker build

Builds a Docker image.

-t reactapp

Assigns the image name:

reactapp

.

Uses the current directory as the Docker build context.

PART 18 – CHECK THE DOCKER IMAGE

Run:

docker images

You should see:

REPOSITORY

reactapp

The Docker image has now been created inside EC2.

PART 19 – RUN THE DOCKER CONTAINER

Run:

docker run -d -p 80:80 reactapp

Explanation

-d

Runs the container in detached/background mode.

-p 80:80

Maps:

EC2 Port 80



PART 20 – CHECK THE RUNNING CONTAINER

Run:

```
docker ps
```

You should see something similar to:

```
0.0.0.0:80->80/tcp
```

This confirms that the container is running.

PART 21 – ACCESS THE REACT APPLICATION

Open a browser on your Windows computer.

Enter:

```
http://<EC2-PUBLIC-IP>
```

Example:

```
http://13.xxx.xxx.xxx
```

Your React application should now appear.

What happened?

Browser



EC2 Public IP



Port 80



Docker Container



React Application

PART 22 – DOCKER CONTAINER LIFECYCLE

Check running containers:

```
docker ps
```

Stop the container:

```
docker stop <CONTAINER-ID>
```

Check:

```
docker ps
```

The container should no longer appear.

Now check all containers:

```
docker ps -a
```

The stopped container should appear.

Remove it:

```
docker rm <CONTAINER-ID>
```

Verify:

```
docker ps -a
```

PART 23 – DOCKER IMAGE VS CONTAINER

Check images:

```
docker images
```

The `reactapp` image should still exist.

Important:

Removing a container does not automatically remove the Docker image.

Docker Image



docker run



Container

Multiple containers can be created from the same image.

METHOD 2 – BUILD DOCKER IMAGE OUTSIDE EC2

Now we will change the deployment process.

Instead of building the Docker image inside EC2, we will build it on the development machine and push it to Docker Hub.

New Architecture

Windows Computer



Docker Build



Docker Image



Docker Hub



AWS EC2



Docker Pull



Docker Container



React Website

PART 24 – BUILD DOCKER IMAGE ON WINDOWS

Open Windows PowerShell.

Navigate to the React project directory.

Example:

```
cd C:\Users\DELL\Desktop\aws-cicd
```

Check the project files:

```
dir
```

Make sure the **Dockerfile** exists.

Build the image:

```
docker build -t reactapp .
```

Check:

```
docker images
```

You should see:

```
reactapp
```

The image now exists on your Windows computer.

PART 25 – LOGIN TO DOCKER HUB

Run:

```
docker login
```

Enter your Docker Hub credentials or access token.

Expected result:

```
Login Succeeded
```

PART 26 – TAG THE DOCKER IMAGE

Docker Hub username:

```
<DOCKERHUB-USERNAME>
```

Example:

rakshith006

Tag the image:

```
docker tag reactapp <DOCKERHUB-USERNAME>/reactapp:latest
```

Example:

```
docker tag reactapp rakshith006/reactapp:latest
```

PART 27 – PUSH THE IMAGE TO DOCKER HUB

Run:

```
docker push <DOCKERHUB-USERNAME>/reactapp:latest
```

Example:

```
docker push rakshith006/reactapp:latest
```

Wait until the upload is complete.

Open Docker Hub and verify that the repository contains:

reactapp

with the tag:

latest

PART 28 – CONNECT TO EC2

From Windows PowerShell:

```
ssh -i "react-deployment.pem" ubuntu@<EC2-PUBLIC-IP>
```

Now we are back inside EC2.

PART 29 – PULL THE IMAGE FROM DOCKER HUB

Inside EC2:

```
docker pull <DOCKERHUB-USERNAME>/reactapp:latest
```

Example:

```
docker pull rakshith006/reactapp:latest
```

Docker downloads the image from Docker Hub into EC2.

PART 30 – CHECK THE IMAGE

Run:

```
docker images
```

You should see:

```
rakshith006/reactapp
```

The image is now stored inside EC2.

PART 31 – RUN THE CONTAINER INSIDE EC2

Run:

```
docker run -d -p 80:80 <DOCKERHUB-USERNAME>/reactapp:latest
```

Example:

```
docker run -d -p 80:80 rakshith006/reactapp:latest
```

Check:

```
docker ps
```

You should see:

```
0.0.0.0:80->80/tcp
```

PART 32 – ACCESS THE APPLICATION

Open:

```
http://<EC2-PUBLIC-IP>
```

The React application should appear.

PART 33 – TROUBLESHOOTING

Check running containers

```
docker ps
```

Check all containers

```
docker ps -a
```

Check container logs

```
docker logs <CONTAINER-ID>
```

Follow logs

```
docker logs -f <CONTAINER-ID>
```

Press:

```
Ctrl + C
```

to exit.

Check Docker images

docker images

PART 34 – CHECK AWS SECURITY GROUP

If the application does not open in the browser:

Go to:

AWS Console → EC2 → Security Groups → Inbound Rules

Make sure HTTP port 80 is allowed.

Required rule:

Type: HTTP

Protocol: TCP

Port: 80

PART 35 – COMPARISON OF BOTH METHODS

Method 1 – Docker Build Inside EC2

GitHub



EC2



git clone



docker build



Docker Image



docker run

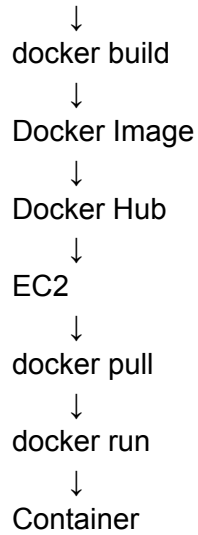


Container

EC2 performs the Docker build.

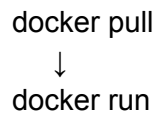
Method 2 – Docker Build Outside EC2

Windows / Development Machine



EC2 does not build the image.

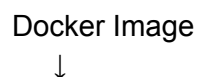
EC2 only:



PART 36 – WHY USE DOCKER HUB?

Docker Hub acts as a registry for Docker images.

Instead of building the same application on every server:



Docker Hub



EC2 EC2 EC2

The same image can be distributed to multiple servers.

Benefits:

- Consistent deployment
- Easy image distribution
- Version management
- Faster deployment
- Reproducible environments

PART 37 – REAL DEVOPS WORKFLOW

The deployment we performed manually today is:

Developer



GitHub



Docker Build



Docker Hub



AWS EC2



Docker Pull



Docker Run



Application

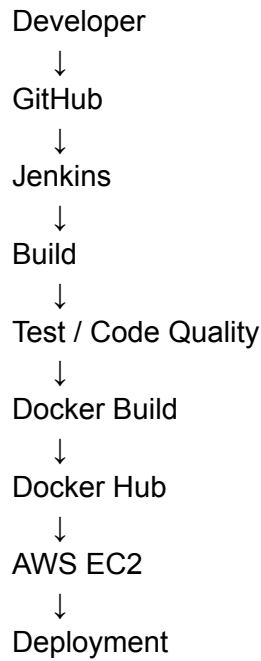
However, all these steps are still manual.

This leads to the need for **CI/CD automation**.

PART 38 – DAY 4 PREVIEW: CI/CD

Tomorrow, we will automate the deployment process using Jenkins.

The future workflow will be:



Instead of manually performing every step, Jenkins will automate the process.

STUDENT HANDS-ON CHALLENGE

Challenge: Deploy a React Application on AWS EC2

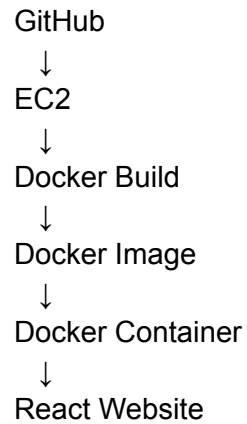
Each student must:

1. Create an EC2 Ubuntu instance.
2. Configure the Security Group.
3. Connect to EC2 using SSH.
4. Install Git.
5. Install Docker.
6. Configure Docker permissions.
7. Clone the React project from GitHub.
8. Build the Docker image inside EC2.
9. Run the Docker container.
10. Access the application using the EC2 Public IP.
11. Stop and remove the container.
12. Build the Docker image on the local machine.
13. Tag the image.
14. Push the image to Docker Hub.
15. Connect to EC2.

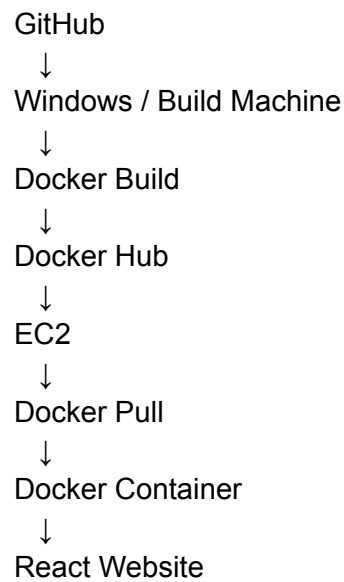
16. Pull the image from Docker Hub.
 17. Run the container inside EC2.
 18. Access the application again using the EC2 Public IP.
-

FINAL DAY 3 ARCHITECTURE

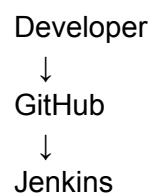
Method 1

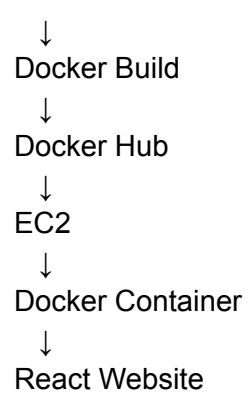


Method 2



Future CI/CD





Key Takeaway

Method 1: EC2 builds and runs the Docker image.

Method 2: The image is built outside EC2, stored in Docker Hub, and then pulled and run inside EC2.

CI/CD: Jenkins will automate the build, image publishing, and deployment process.